# Evolutionary rescue under habitat fragmentation

Jan Leuenberger & Safeeya Ramis

2026-10-28

## Abstract

Habitat fragmentation is a major threat to wildlife populations and can occur after ER has already begun, yet its effect on this process is not well understood. We extended the Orr & Unckless model into a discrete-time stochastic simulation of a haploid population with logistic growth. At a defined generation, the habitat was split into two fragments, and we varied the timing of the split, the relative habitat size, the distribution of individuals, and the migration rate. For each scenario, we estimated rescue probability from 100 replicates. ER remained possible after fragmentation, and rescue probability tended to increase with later splits, although this may be inflated because only populations that survived until the split were included. When migration was possible, smaller habitats were rescued more often than larger ones, whereas the distribution of individuals had little effect. Migration strongly influenced outcomes, but we found no evidence that it improved rescue overall. These patterns may partly reflect asymmetric migration and our rescue definition rather than true differences in adaptation. Further simulations are needed before these findings can inform conservation measures.

## Introduction

The Anthropocene is characterized by a wide variety of changes and threats that affect nearly all wildlife populations. These human-induced pressures are having an increasingly negative impact on the stability and growth of populations and are leading to a continuous decline in the individual numbers within the populations. In times like these, the idea of **evolutionary rescue** (ER) is becoming increasingly important. *"ER occurs when genetic adaptation allows a population to recover from demographic effects initiated by environmental change that would otherwise cause extirpation"* (Gonzalez et al., 2013) and therefore leads back to an increase in the number of individuals within a population.

Because ER is often viewed as a solution to environmental change and pressure, we thought about what influence a major environmental threat such as **habitat fragmentation**; *"the process by which habitats are transformed into smaller patches isolated from each other [...]"* (Cheptou et al., 2017) might have on an ER process that has already begun.

### Research Questions

Based on this reflection, the following questions emerged, which we intended to explore in more detail as part of a three-week theoretical practical course at the University of Bern:

- Is evolutionary rescue possible in fragmented habitats if the habitats are separated at a certain time point after the ER processes have already begun?
  - How does the timing of habitat fragmentation affect evolutionary rescue?

- How does the size of the habitats affect evolutionary rescue?
- What impact does the distribution of individuals across habitats have on evolutionary rescue?
- What would happen if migration between the isolated fragmented habitats were possible?

It is important to analyze these questions in more detail because habitat fragmentation represents an increasing threat to populations, which can occur at any time. Furthermore, from a conservation perspective, it is important to understand the impact of splitting time, habitat size, and individual distribution on evolutionary rescue in order to ensure the survival of the populations.

This report will document a stochastic simulation of **evolutionary rescue** in discrete time, based on the model of Orr & Unckless introduced in the practical, which was extended with different parameters to answer the questions listed above. Therefore, an explanation of the model is provided in the Methods section, followed by a presentation of the results obtained and a subsequent discussion.

# Methodology

## Base Model

The base model of Orr & Unckless, which was provided from the banklab recreating a biological scenario on which we built on. In this scenario a haploid population consists of wild-type individuals (**a**) and, possibly, mutant individuals (**A**). A sudden environmental change has a negative efefect on the wild type and makes it unfit, which is why it starts to **decline**. The mutant allele is **beneficial** and is able to grow under the new environmental conditions, but it is initially rare or absent. If it's absent, it has to arise first by mutation from the wild type. The population is then **rescued** if the mutant spreads before the wild type goes extinct.

## Fragmentation and migration

To assess the impact of **habitat fragmentation** on ER, we extended the model by adding a additional biological scenario, where the habitat, in which the population is located (called habitat A), gets separated after a defined number of generations into **two isolated habitats** (called habitat B and C) of defined sizes. Based on that, the following equation must be considered with regard to the habitat sizes:($A_size = B_size + C_size$). Each habitat has also its own carrying capacity (**K**) related to its size: ($K_A = K_B + K_C$). At a **separation time** individuals of the population A are distributed then into the two new habitats B and C and both will then evolve independently. Since our aim was also to investigate what happens when **migration** is possible, we allowed individuals to move between habitats B and C after the splitting event. In each generation, every individual (wild type and mutant separately) migrates to the other habitat with probability **migrate**.

## Parameters and variables

Overall, we have expanded the base model therefore with five parameters; `split_time`, `habitat_prop`, `split_ratio`, `K` and `migration`. All the other parameters and variables were already set in the based model. The table below provides brief explanations of the respective parameters and variables.

**Taken from the base model:**

| Name | Meaning |
|---|---|
| `N_a`, `N_A` | current size of the wild-type / mutant population (change over time) |

| Name | Meaning |
|---|---|
| `N_init_a`, `N_init_A` | initial sizes of wild type (positive integer) and mutant (non-negative integer) |
| `decay_rate` | detrimental effect of the environment on the wild type ($> 0$ so that extinction is possible) |
| `sel_coeff` | selective advantage of the mutant (should be $>$ `decay_rate` to allow rescue) |
| `mut_rate` | mutation rate from wild type to mutant ($> 0$ and small) |
| `t_max` | maximum number of generations simulated (a setting, not a model parameter) |

**Added to expand the model:**

| Name | Meaning |
|---|---|
| `split_time` | generation at which the fragmentation happens (integer $> 0$) |
| `habitat_prop` | defining size of the new habitat B and therefore also it's carrying capacity (between 0 and 1); habitat C gets `1 - habitat_prop` |
| `split_ratio` | probability with which each individual is assigned to habitat B at the split (between 0 and 1): habitat C gets `1 - split_ratio` |
| `K` | carrying capacity of the original (unsplit) habitat; limits growth and is divided among B and C in the same proportions as the habitat size |
| `migrate` | migration rate: (between 0 and 1) with which each individual moves to the other habitat in every generation after the split. |

**Example**

To ensure that there is no confusion and that the parameters and variables are clear, here is a brief example:

| Parameters | values |
|---|---|
| `N_a`, `N_A` | 200, 100 |
| `split_time` | 30 |
| `habitat_prop` | 0.8 |
| `split_ratio` | 0.7 |
| `K` | 1000 |
| `migrate` | 0.01 |

The carrying capacity of habitat A is set at 1000 individuals. Based on the `habitat_prop` we know that habitat B is able to carry 800 individuals ($K * habitat_{prop}$) and habitat C is able to carry 200 individuals ($K * (1 - habitat_{prop})$). From the `split_ratio` we know that 140 wild type individuals and 70 mutant individuals are located within the habitat B ($N_a * split_{time}$) & ($N_A * split_{time}$). In habitat C we will then be able to find 60 wild type individuals and 30 mutant individuals ($N_a * (1 - split_{time})$ & ($N_A * (1 - split_{time})$). With `migrate` = 0.01, each individual has a 1% chance per generation of moving to the other habitat.

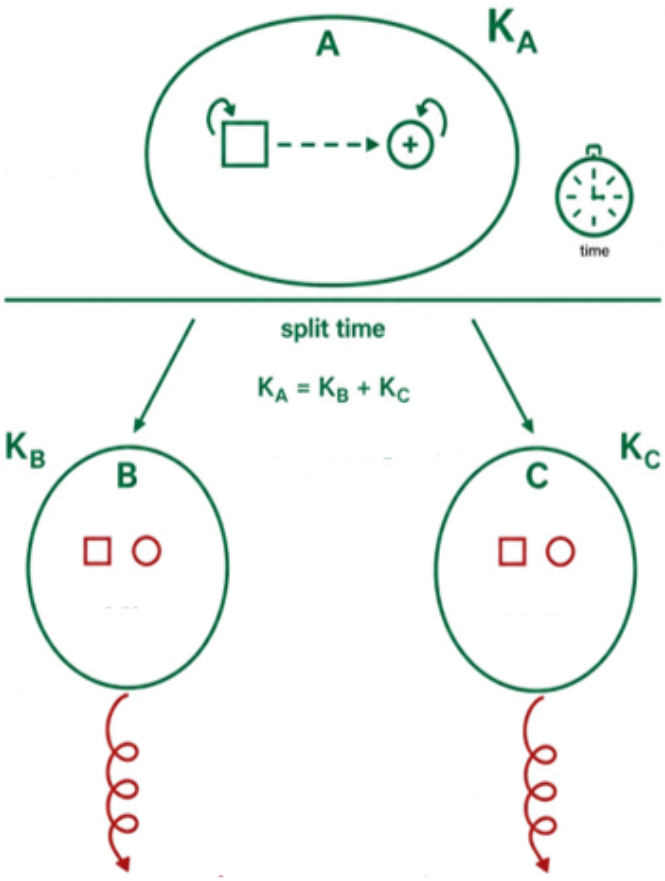



Figure 1: *Simplified sketch of the model*

## Model/Simulation

### Reproducibility

To ensure the reproducibility of our results, it is important to set a seed first, in our case 57. In the same chunk we installed the libraries needed to run our simulation and analysis.

### Overview of the code

1. `simulate_one_gen()`: simulates **one generation** (reproduction with logistic growth + mutation) for one habitat.
2. `simulate_pop()`: runs a **full simulation**: one habitat until `split_time`, then two habitats (B and C) afterwards.
3. Plot functions (`plot_split()`, `plot_habitat()`, `plot_habitats()`): visualise a single simulation run.
4. `run_scenarios()` and `to_long()`: repeat the simulation many times for different parameter values and summarise the **rescue probability**.
5. Six analyses: split time varies, habitat proportion varies and migration rates

### Simulate one generation

Our actual simulation begins in the second chunk, although the change in population is first calculated only within a single generation. To do this, we use the function `simulate_one_gen()`, which simulates **one generation** based on the following steps:

1. **Expected offspring (logistic growth)**

   Without competition, the per-capita growth rates of the individual are:

   - wild type: $\rho_a = -r$ (negative, so the type declines), with $r =$ `decay_rate`
   - mutant: $\rho_A = s - r$ (positive if $s > r$), with $s =$ `sel_coeff`

   In addition a carrying capacity is added by a **density-dependent term**, means that the growth factor of each type is reduced by $\rho_A \cdot N_{tot}/K$. The expected number of offspring is therefore

   $$\bar{N}_a = N_a \max\!\big(0,\ 1 + \rho_a - \rho_A \tfrac{N_{tot}}{K}\big)\,, \qquad \bar{N}_A = N_A \max\!\big(0,\ 1 + \rho_A - \rho_A \tfrac{N_{tot}}{K}\big)$$

   When $N_{tot}$ approaches $K$, the mutant's growth factor approaches 1 (population stays constant) and the wild type keeps declining. When $N_{tot} > K$ the mutant would shrink back towards $K$. The `max(0, ...)` prevents negative expected values, which `rpois()` cannot handle.

2. **Demographic stochasticity**

   The actual number of offspring is drawn from a **Poisson distribution** whose mean is the expected value above. This is what makes every run different.

3. **Mutations**

   Of the wild-type offspring (a), a Poisson-distributed number (mean `offsp_a * mut_rate`) mutates to mutant individuals (A). These are subtracted from the wild type and added to the mutants.

The function then returns a vector `c(N_a_new, N_A_new)`, containing the current population size based on these calculations.

Since the base model describes exponential growth and this needed to be converted to logistic growth in this model, we used AI to some extent to adapt the code for this purpose.

```r
simulate_one_gen <- function(N_a, N_A, decay_rate, sel_coeff, mut_rate, K) {

  # total population size and per-capita growth rates without competition
  N_tot <- N_a + N_A
  rho_a <- -decay_rate
  rho_A <- sel_coeff - decay_rate
  # logistic growth: crowding reduces the growth of both types equally (set to 0 if negative)
  mean_a <- N_a * max(0, 1 + rho_a - rho_A * N_tot / K)
  mean_A <- N_A * max(0, 1 + rho_A - rho_A * N_tot / K)
  # draw offspring according to Poisson distribution
  offsp_a <- rpois(1, mean_a)
  offsp_A <- rpois(1, mean_A)
  # draw new mutants according to Poisson distribution
  mut_a_to_A <- rpois(1, offsp_a * mut_rate)
  # determine new population sizes of wild type and mutant
  N_a_new <- offsp_a - mut_a_to_A
  N_A_new <-  offsp_A + mut_a_to_A

  return(c(N_a_new, N_A_new))
}
# Test the function
#print(simulate_one_gen(100,0,0.1,0.2,0.001,1000))
```

### Simulating the whole populations

The function `simulate_one_gen()` describes a single generation, whereas evolutionary rescue unfolds over many. We therefore iterate this step in `simulate_pop()`, which integrates habitat fragmentation (the population is divided into two isolated habitats at a chosen generation) and migration (individuals may move between these habitats afterwards). The function records the numbers of wild-type and mutant individuals in each habitat, so we can determine whether the mutant spreads and rescues the population, and how the timing of the split, the relative habitat sizes and the migration rate influence this outcome.

The function `simulate_pop()` runs the full simulation over `t_max` generations and has three phases:

1. **generations (i) $<$ `split_time + 1`** In this first phase, the population lives in one habitat with carrying capacity K, and each generation is simulated with `simulate_one_gen()`. This phase stops early if the population goes extinct or grows to at least twice its initial size.

2. **generation (i) $==$ `split_time + 1`** In the second phase, at generation `split_time + 1`, the habitat is fragmented into two isolated habitats, B and C. The habitat sizes are calculated based on the `habitat_prop`, as explained above. The existing individuals are distributed as well based on the `split_ratio` using `rbinom()`.

3. **generation (i) $>$ `split_time + 1`** From then on both habitats are simulated independently, each with its own carrying capacity. After each generation, migration can occur, where individual moves to the other habitat with probability migrate, drawn separately for wild types and mutants. Migration is symmetric in its rate but not in its effect. The number of migrants is proportional to the size of the source habitat and migration is possible in both directions.

It's important to note that `spit_time` must be equal to 1, because in the for loop, `i` isn't counted the same way as our number of generations, but is shifted by one. Furthermore, the three phases mentioned above are not represented in the same order in the code. However, this does not affect the calculations, since the if statements check the number of past generations.

At the end the function returns a list of three data frames: main (habitat A), B and C, containing the population sizes for each habitat. If the population dies out or reaches twice its starting size before the split, B and C are empty.

```
simulate_pop <- function(N_init_a, N_init_A, decay_rate, sel_coeff, mut_rate, t_max, K, habitat_prop, sp
  # Create the vector in which to save the results
  pop_vector <- c(N_init_a, N_init_A)
  # initiate the variables
  pop_new <- c(N_init_a, N_init_A)
  # Create the vectors for the new habitats
  pop_vector_B <- NULL
  pop_vector_C <- NULL
  # run the simulation until generation t_max
  for (i in 1:t_max) {
    # Time when the fragmentation occur
    if (i == split_time + 1) {
      # Create the new habitats (creating new K)
      K_B <- K * habitat_prop
      K_C <- K * (1 - habitat_prop)
      #create the ratio of the splitting

      split_ratio_B <- split_ratio
      split_ratio_C <- (1 - split_ratio_B)

      # Distribute individuals  randomly to the habitat B
      n_B_a <- rbinom(1, pop_new[1], split_ratio_B)
      n_B_A <- rbinom(1, pop_new[2], split_ratio_B)

      # Distribute individuals randomly to the habitat C
      n_C_a <- pop_new[1] - n_B_a
      n_C_A <- pop_new[2] - n_B_A

      # Calculate the new populations for B and C
      pop_new_B <- simulate_one_gen(n_B_a, n_B_A, decay_rate, sel_coeff, mut_rate, K_B)
      pop_new_C <- simulate_one_gen(n_C_a, n_C_A, decay_rate, sel_coeff, mut_rate, K_C)

      # add the new population sizes to the output vector
      pop_vector_B <- rbind(pop_vector_B,pop_new_B)
      pop_vector_C <- rbind(pop_vector_C,pop_new_C)
    }

    if (i > split_time + 1) {

      pop_new_B <- simulate_one_gen(
        pop_new_B[1], pop_new_B[2],
        decay_rate, sel_coeff, mut_rate,K_B)

      pop_new_C <- simulate_one_gen(
        pop_new_C[1], pop_new_C[2],
        decay_rate, sel_coeff, mut_rate,K_C)

      # Migration
      migrateB_vec <- c(rbinom(1, pop_new_B[1], migrate),
                        rbinom(1, pop_new_B[2], migrate))
```

```
      migrateC_vec <- c(rbinom(1, pop_new_C[1], migrate),
                        rbinom(1, pop_new_C[2], migrate))

      #adding the migrating individuals back to the Populations
      pop_new_B <- pop_new_B - migrateB_vec + migrateC_vec
      pop_new_C <- pop_new_C - migrateC_vec + migrateB_vec

      # Save the results for each generation
      pop_vector_B <- rbind(pop_vector_B, pop_new_B)
      pop_vector_C <- rbind(pop_vector_C, pop_new_C)

      # define the row and column names of the output vector
      rownames(pop_vector_B) <- (0:t_max)[1:length(pop_vector_B[,1])]
      rownames(pop_vector_C) <- (0:t_max)[1:length(pop_vector_C[,1])]
      colnames(pop_vector_B) <- c("a","A")
      colnames(pop_vector_C) <- c("a","A")

      if (sum(pop_new_B) + sum(pop_new_C) >= 2.5*(N_init_a + N_init_A) |
          sum(pop_new_B) + sum(pop_new_C) == 0) break
    }

    # Continue simulating habitat A only until split_time
    if (i < split_time + 1) {

      # redefine the current population one generation later
      pop_new <- simulate_one_gen(pop_new[1],pop_new[2], decay_rate, sel_coeff, mut_rate,K)
      # add the new population sizes to the output vector
      pop_vector <- rbind(pop_vector,pop_new)
      # condition to stop the simulation before t_max: either the population exceeds 2 times the origin
      if (pop_new[1]+pop_new[2]>=2*(N_init_a+ N_init_A) | pop_new[1]+pop_new[2]==0) break
    }

  }
  # define the row and column names of the output vector
  rownames(pop_vector) <- (0:t_max)[1:length(pop_vector[,1])] # note that the vector has to be cut if t
  colnames(pop_vector) <- c("a","A")
  # Data frames
  pop_vector_df <- as.data.frame(pop_vector)
  pop_vector_B_df <- as.data.frame(pop_vector_B)
  pop_vector_C_df <- as.data.frame(pop_vector_C)
  # return all three population vectors separately
  return(list(main = pop_vector_df, B = pop_vector_B_df, C = pop_vector_C_df))
}
```

### Plotting functions for the model

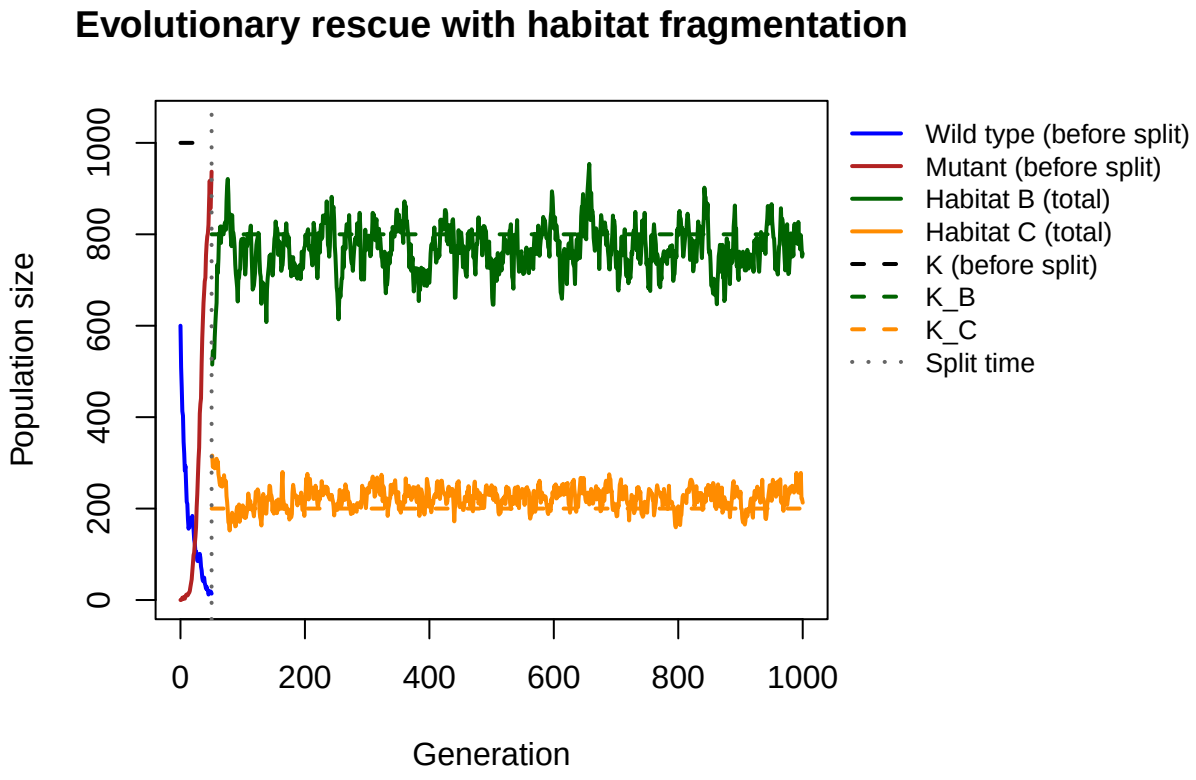
To inspect what happens in a single simulation, we define three plot functions. plot_split() shows the whole run: wild type and mutant before the split, and the total population of habitats B and C afterwards, together with the carrying capacities (K, K_B, K_C) and the split time. plot_habitat() draws one habitat in detail, with the wild type, the mutant and the total population, and plot_habitats() stacks the panels of B and C so that they can be compared directly. These plots serve to check that the simulation behaves as expected, and they were used extensively for debugging in the early stages of the project. We used base R graphics here, and the plots received very positive feedback. Opinions on whether to use base R or ggplot2 for plotting are

divided, even within the **THEE** Lab. In the later plotting functions we use ggplot2, both as an exercise to learn the package and to show that both approaches can produce informative figures.

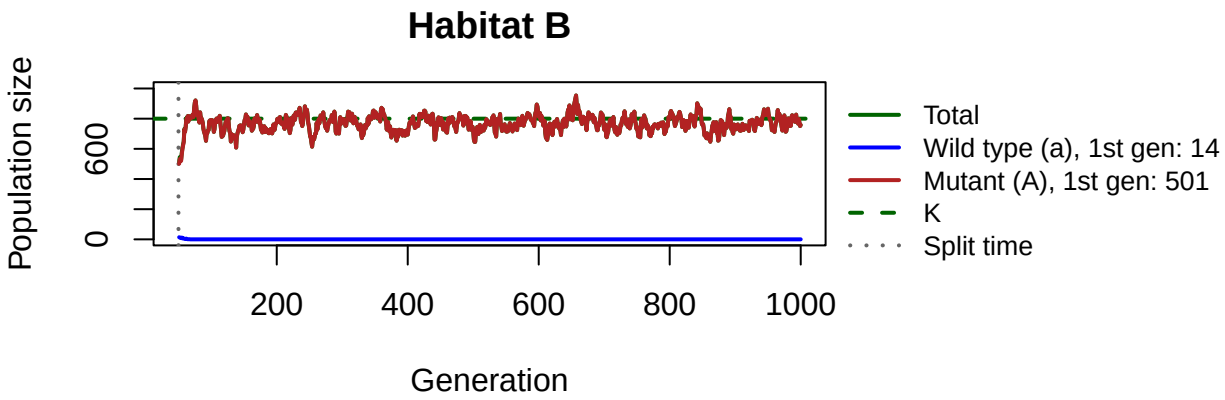
```
# ============================================================
# Usage
# (simulate_one_gen and simulate_pop must be defined first)
# ============================================================
# set parameters
max_gen      <- 1000
K            <- 1000
habitat_prop <- 0.8
split_time   <- 50
split_ratio  <- 0.5
migrate      <- 0.01

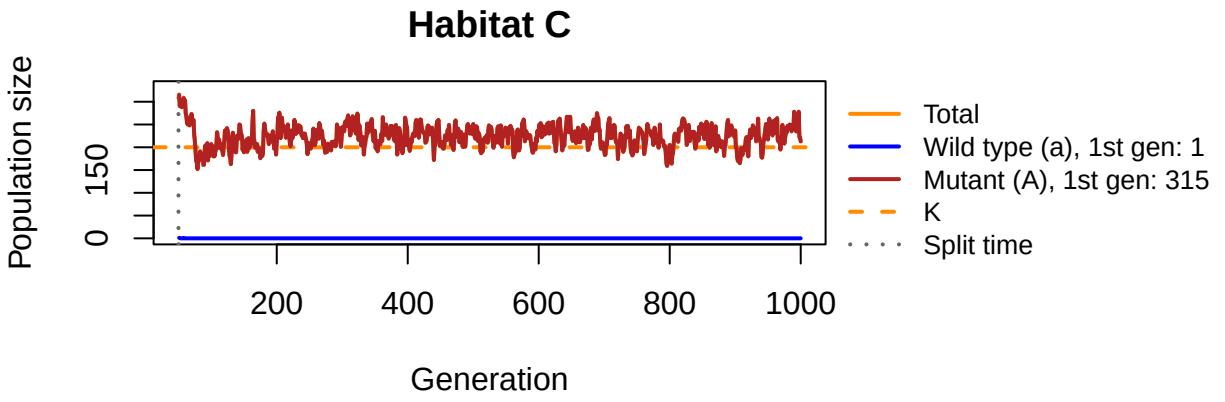
# run simulation and define it as output
output <- simulate_pop(600, 0, 0.01, 0.2, 0.001, max_gen, K,
                       habitat_prop, split_ratio , split_time,migrate)

# overview plot
plot_split(output, K = K, habitat_prop = habitat_prop, split_time = split_time)
```

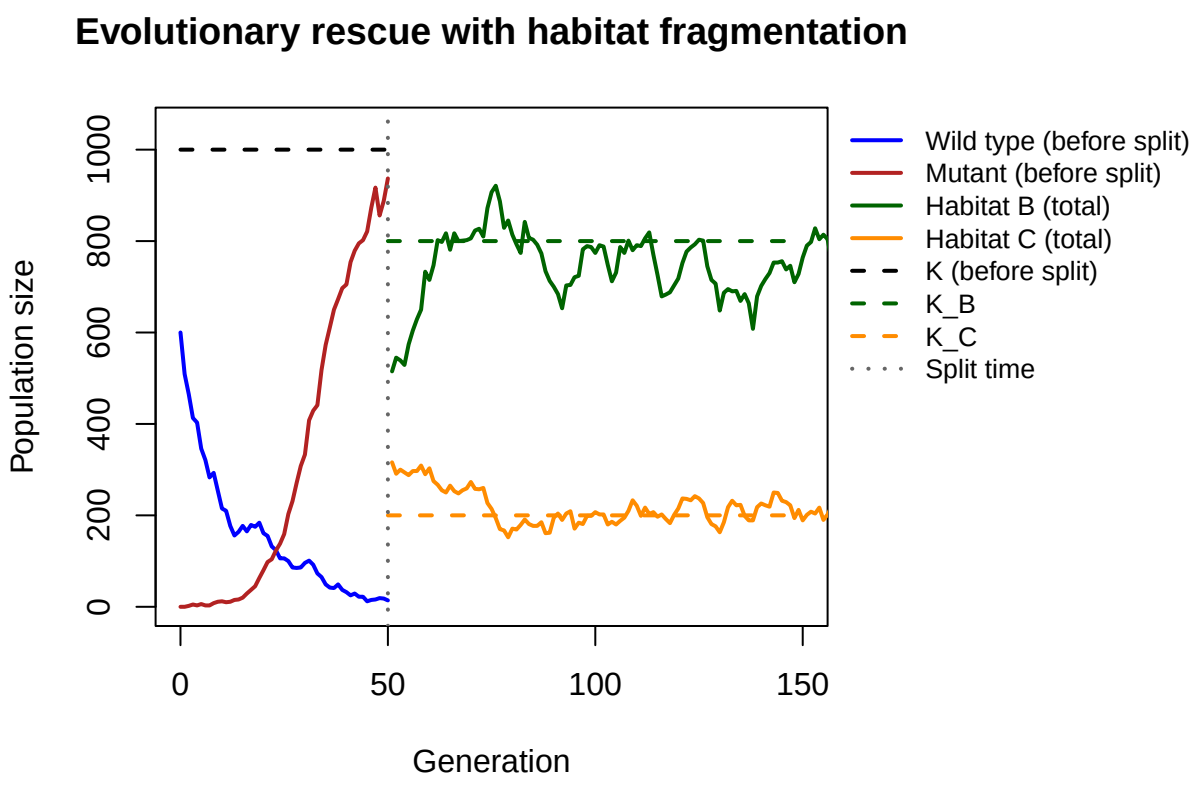



```
# panels for habitat B and C
plot_habitats(output, K = K, habitat_prop = habitat_prop, split_time = split_time)
```

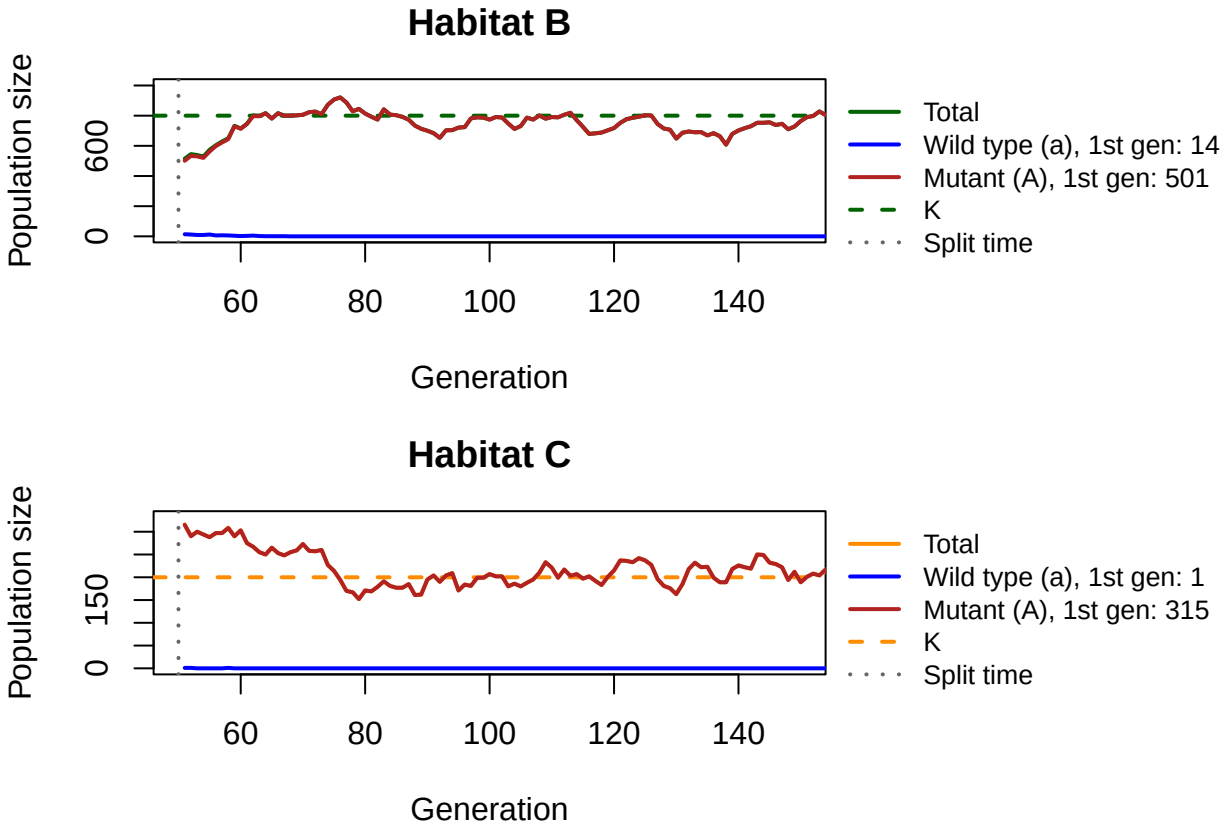



```
# zoomed in on the first 150 generations
plot_split(output, K = K, habitat_prop = habitat_prop,
           split_time = split_time, xlim_max = 150)
```




```
plot_habitats(output, K = K, habitat_prop = habitat_prop,
              split_time = split_time, xlim_max = 150)
```







## Analyses and Results

A single simulation is one random realization of the model, so we repeated each scenario and report the rescue probability, the proportion of replicates in which rescue occurred. Only the parameters named in each analysis are varied. Migration is switched on in analyses 1 to 5 (migrate = 0.01) and varied only in the last analysis.

#### What `run_scenarios()` does

The function `run_scenarios()` loops over every combination of split time, habitat proportion, split ratio and migration rate, and each combination is treated as one scenario. For each scenario it calls `simulate_pop()` repeatedly until 100 replicates have been made. Runs in which the population went extinct (or reached twice its initial size) before the split are discarded and drawn again, because habitats B and C do not exist in these runs.

The helper function `to_long()` reshapes the summary for `ggplot2`: the columns `rescued_B`, `rescued_C` and `rescued_both` are combined into a single column, and the standard error of a proportion, $\sqrt{p(1-p)/n}$, is added and shown as error bars in the plots.

#### Important points for interpretation

Three features affect interpretation. First, rescue probabilities are conditional on the population still existing at the split. Second, a habitat counts as rescued only if the number of mutants in the final generation is at

least its K. Third, with 100 replicates, the standard error is up to about 0.05, so small differences should not be interpreted as real effects.

## Overview of the analysis code

```
# fixed parameters
init_a           <- 600
init_A           <- 0
m_rate           <- 0.001
max_gen_analysis <- 1000
K_analysis       <- 1000
no_replicates    <- 100    # more replicates give more precise probabilities
max_attempts     <- 5000
rescue_frac      <- 1
migrate          <- 0.01    # rescue = A >= rescue_frac * K of the habitat, changeable

# core function from analyse section of base model
run_scenarios <- function(split_times, habitat_props, split_ratios,migrates = migrate) {
  results <- list()

  # extend analysis with split_time, habitat_prop, split_ratio and migration
  for (st in split_times) {
    for (hp in habitat_props) {
      for (sr in split_ratios) {
        for (mg in migrates) {

        # Habitat sizes
        K_B <- K_analysis * hp
        K_C <- K_analysis * (1 - hp)

        #set to zero
        i <- 0
        attempts <- 0

        repeat {
          attempts <- attempts + 1

          if (attempts > max_attempts) {
            warning(paste(
              "Too many failed runs for st =", st,
              "hp =", hp,
              "sr =", sr,
              "mig =", mg
            ))
            break
          }

          one_run <- simulate_pop(
            init_a, init_A,
            decay_rate   = 0.1,
            sel_coeff    = 0.2,
            mut_rate     = m_rate,
```

```
          t_max        = max_gen_analysis,
          K            = K_analysis,
          habitat_prop = hp,
          split_ratio  = sr,
          split_time   = st,
          migrate      = mg
        )

        # Skip runs that ended before the split time
        if (nrow(one_run$B) == 0 || nrow(one_run$C) == 0) next

        i <- i + 1

        # Populations reaching carrying capacity
        reached_B <- one_run$B$A >= rescue_frac * K_B
        reached_C <- one_run$C$A >= rescue_frac * K_C

        rescued_B <- reached_B[length(reached_B)]
        rescued_C <- reached_C[length(reached_C)]

        # summerizig results
        results[[length(results) + 1]] <- data.frame(
          st = st,
          hp = hp,
          sr = sr,
          mig = mg,
          replicate = i,
          rescued_B = rescued_B,
          rescued_C = rescued_C,
          rescued_any = rescued_B | rescued_C,
          rescued_both = rescued_B & rescued_C,
          gen_B = if (rescued_B) which(reached_B)[1] else NA_integer_,
          gen_C = if (rescued_C) which(reached_C)[1] else NA_integer_
        )

        if (i >= no_replicates) break
       }
      }
    }
  }
}

# Combine all simulation results
if (length(results) == 0) {
  stop("No valid simulation results were obtained.")
}

# Create data table
data_table <- do.call(rbind, results)

# Rescue probability per scenario
rescue_prob <- data_table %>%
  dplyr::group_by(st, hp, sr, mig) %>%
```

```
    dplyr::summarise(
      n = dplyr::n(),
      dplyr::across(
        c(rescued_B, rescued_C, rescued_any, rescued_both),
        mean
      ),
      .groups = "drop"
    )

  # Define what will be returned
  return(list(
    data = data_table,
    prob = rescue_prob
  ))
}

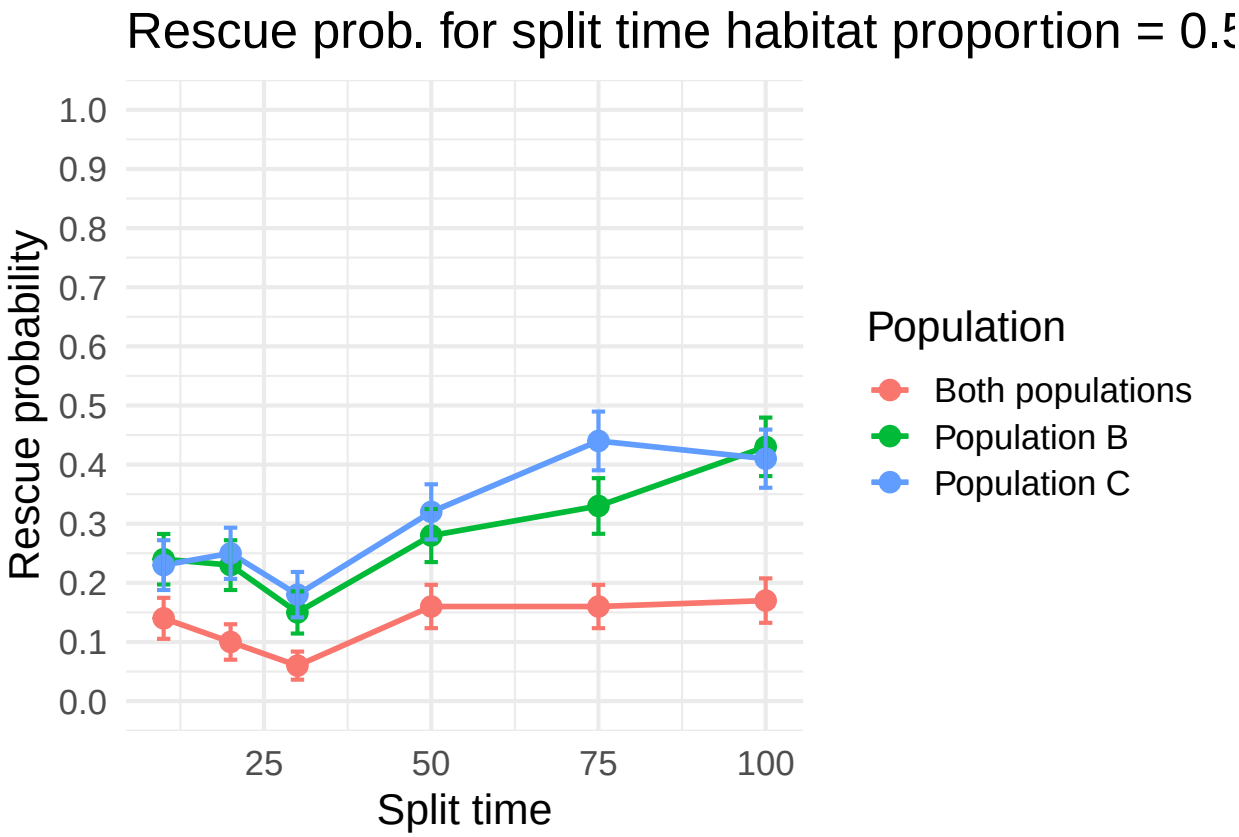

# ---- helperfunction : long format with standard error for plotting ----

to_long <- function(rescue_prob) {
  rescue_prob %>%         #%>% is the pipe operator. It takes whatever is on its left and passes it as
    tidyr::pivot_longer(   #doing this for ggplot 2
      cols = c(rescued_B, rescued_C, rescued_both),
      names_to = "population",
      values_to = "rescue_probability"
    ) %>%
    dplyr::mutate(
      #standard error of a proportion
      se = sqrt(rescue_probability * (1 - rescue_probability) / n),
      population = dplyr::recode(
        population,
        rescued_B    = "Population B",
        rescued_C    = "Population C",
        rescued_both = "Both populations"
      )
    )
}
```

```
# ========================================================================
# ANALYSIS 1: split time varies, habitat proportion fixed
# ========================================================================
# run analysis with given parameters and store as res1
res1 <- run_scenarios(
  split_times   = c(10, 20, 30, 50, 75, 100),
  habitat_props = 0.5,
  split_ratios = 0.5
)
# save as long1
long1 <- to_long(res1$prob)
print(res1$prob)

# plot results
ggplot(long1, aes(x = st, y = rescue_probability,
```

```
                color = population, group = population)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = pmax(0, rescue_probability - se),
                    ymax = pmin(1, rescue_probability + se)),
                width = 2) +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.1)) +
  labs(title = "Rescue prob. for split time habitat proportion = 0.5",
       x = "Split time", y = "Rescue probability", color = "Population") +
  theme_minimal(base_size = 16)
```



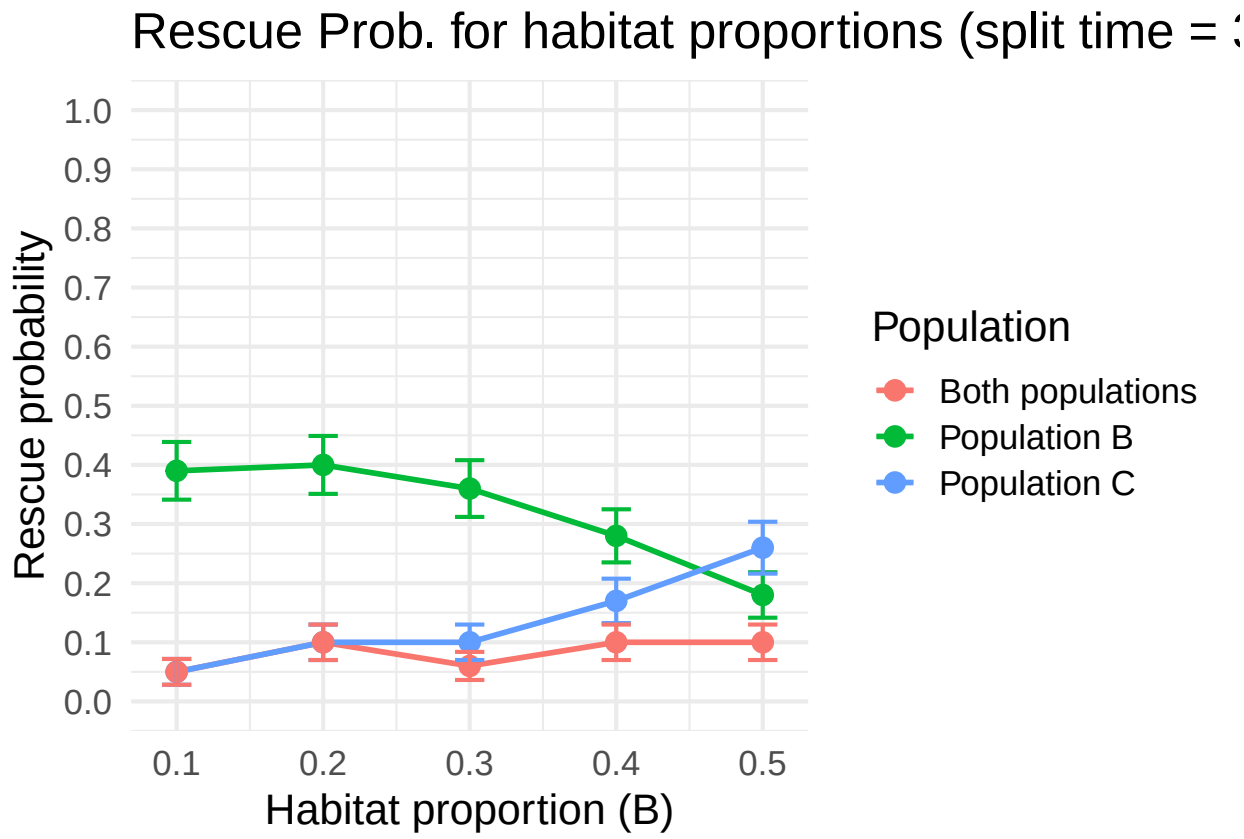

### Timing of fragmentation

This plot shows the proportion of 100 replicate simulations in which the mutant reached the rescue threshold (100% of K), depending on the generation at which the habitat was split into two equal halves. Rescue probability tended to increase with later splits, from about 0.24 (B) and 0.23 (C) at generation 10 to about 0.43 and 0.41 at generation 100, with a dip at generation 30. Habitats B and C behaved very similarly, as expected for identical halves. The probability that both were rescued was lower (about 0.06 to 0.17) and stayed roughly flat from generation 50 onwards.

```
# ======================================================================
# ANALYSIS 2: habitat proportion varies, split time fixed
# ======================================================================
res2 <- run_scenarios(
```

```
  split_times   = 30,
  habitat_props = c(0.1, 0.2, 0.3, 0.4, 0.5),
  split_ratios = 0.5
)
long2 <- to_long(res2$prob)
print(res2$prob)

ggplot(long2, aes(x = hp, y = rescue_probability,
                  color = population, group = population)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = pmax(0, rescue_probability - se),
                    ymax = pmin(1, rescue_probability + se)),
                width = 0.02) +
  scale_x_continuous(breaks = seq(0.1, 0.9, 0.1)) +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.1)) +
  labs(title = "Rescue Prob. for habitat proportions (split time = 30)",
       x = "Habitat proportion (B)", y = "Rescue probability",
       color = "Population") +
  theme_minimal(base_size = 16)
```




### Habitat proportion

This Figure shows how rescue probability changes when the habitat is split differently at generation 30. The x-axis gives the share of the original habitat and also of K that goes to habitat B, with habitat C receiving

the remainder, while individuals were still divided evenly. The smaller habitat was rescued more often. At a habitat proportion of 0.1, habitat B (10% of K) was rescued in 39% of the 100 replicates and habitat C (90% of K) in only 5%. As B grew towards equal size, its rescue probability fell (0.18 at 0.5) and that of C rose (0.26 at 0.5). The probability that both habitats were rescued stayed low at 0.05 to 0.10.

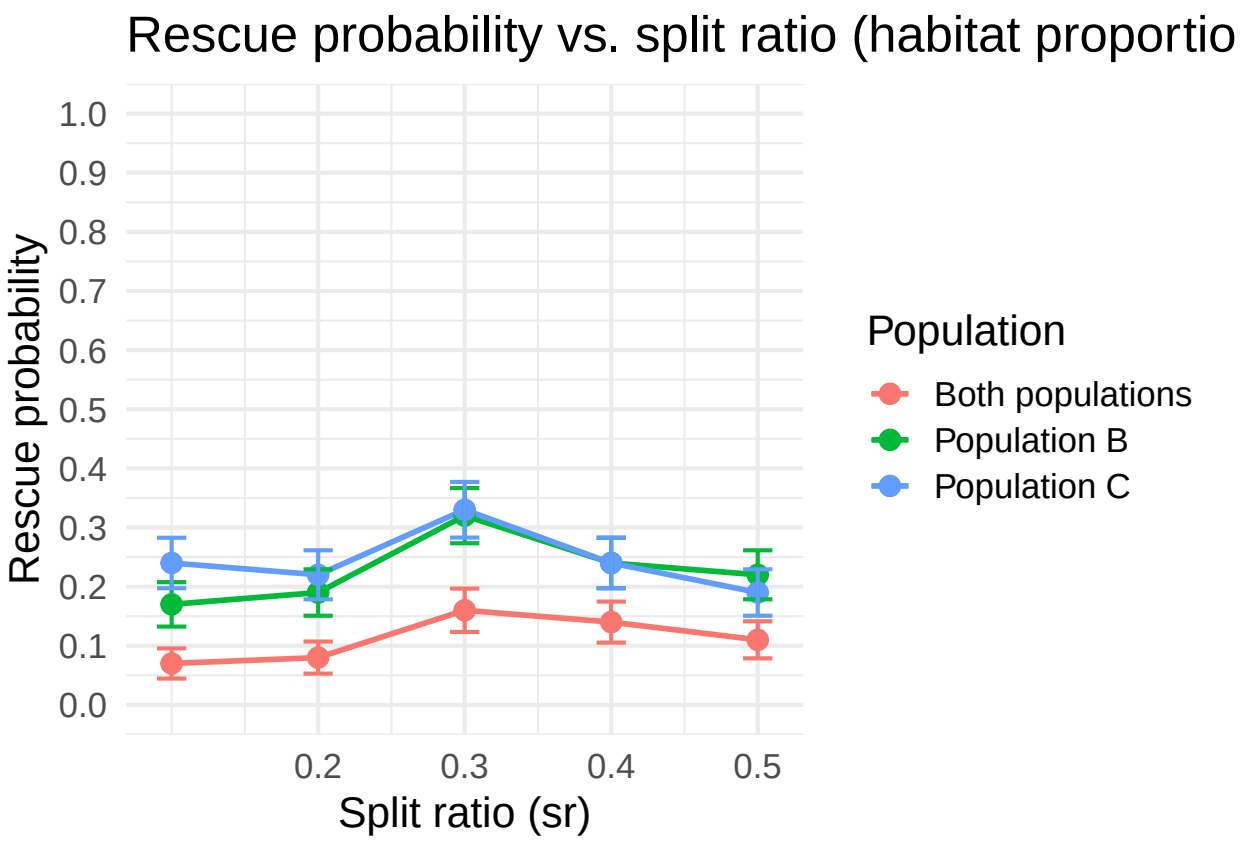
```
# ========================================================================
# ANALYSIS 3: split ratio varies, habitat proportion fixed
# ========================================================================

res3 <- run_scenarios(
  split_times   = 30,  # fix split time
  habitat_props = 0.5,
  split_ratios  = c(0.1, 0.2, 0.3, 0.4, 0.5)
)

long3 <- to_long(res3$prob)

print(res3$prob)

ggplot(long3, aes(x = sr, y = rescue_probability,
                  color = population, group = population)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  geom_errorbar(
    aes(
      ymin = pmax(0, rescue_probability - se),
      ymax = pmin(1, rescue_probability + se)
    ),
    width = 0.02
  ) +
  scale_x_continuous(breaks = seq(0.2, 0.8, 0.1)) +
  scale_y_continuous(
    limits = c(0, 1),
    breaks = seq(0, 1, 0.1)
  ) +
  labs(
    title = "Rescue probability vs. split ratio (habitat proportion = 0.5)",
    x = "Split ratio (sr)",
    y = "Rescue probability",
    color = "Population"
  ) +
  theme_minimal(base_size = 16)
```




### Distribution of individuals

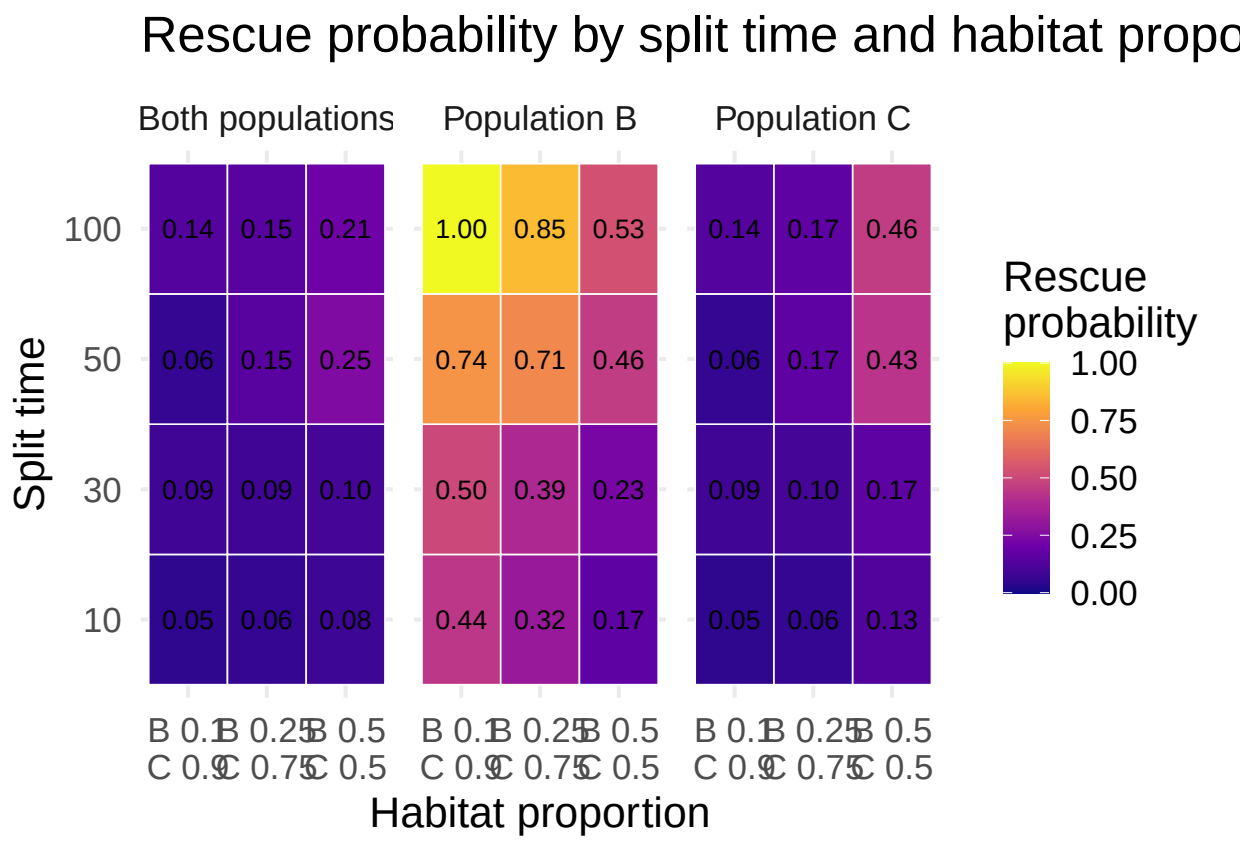
the third analyses shows the effect of how individuals are distributed at the split, with habitats of equal size split at generation 30. The x-axis gives the probability (split ratio) that an individual is assigned to habitat B, with the remaining individuals going to C. Rescue probability showed no consistent trend across split ratios. Habitat B was rescued in 17% to 32% of the 100 replicates and habitat C in 19% to 33%, and the two habitats did not differ clearly at any ratio. Both curves peaked at a split ratio of 0.3. The probability that both habitats were rescued was lower at more uneven splits.

```
# ======================================================================
# ANALYSIS 4: both vary (full grid)
# ======================================================================
res4 <- run_scenarios(
  split_times   = c(10, 30, 50, 100),
  habitat_props = c(0.1, 0.25, 0.5),
  split_ratios = 0.5
)
long4 <- to_long(res4$prob)
print(res4$prob)

# heatmap: one panel per population
# label showing the proportion of BOTH habitats on two lines, e.g. "B 0.1" over "C 0.9"
hp_values <- sort(unique(long4$hp))
long4 <- long4 %>%
  mutate(hp_label = factor(hp, levels = hp_values,
```

```
                          labels = paste0("B ", hp_values, "\nC ", 1 - hp_values)))

ggplot(long4, aes(x = hp_label, y = factor(st), fill = rescue_probability)) +
  geom_tile(color = "white") +
  geom_text(aes(label = sprintf("%.2f", rescue_probability)), size = 3.5) +
  facet_wrap(~ population) +
  scale_fill_viridis_c(limits = c(0, 1), option = "C") +
  labs(title = "Rescue probability by split time and habitat proportion",
       x = "Habitat proportion", y = "Split time",
       fill = "Rescue\nprobability") +
  theme_minimal(base_size= 16)
```




### Split time x habitat proportion

This heatmap combines both factors for habitat proportions of 0.1, 0.25 and 0.5. Rescue of habitat B increased with split time at every proportion. It was highest when B was smallest: at a proportion of 0.1 it rose from 0.44 to 1.00, compared with 0.17 to 0.53 at 0.5. Habitat C, the larger habitat in unequal splits, was rarely rescued (at most 0.17). Only at equal size did its rescue probability increase with split time (0.13 to 0.46). Rescue of both habitats stayed low (0.05 to 0.25). The smaller habitat was thus rescued more often than the larger one. This difference grew with later splits.

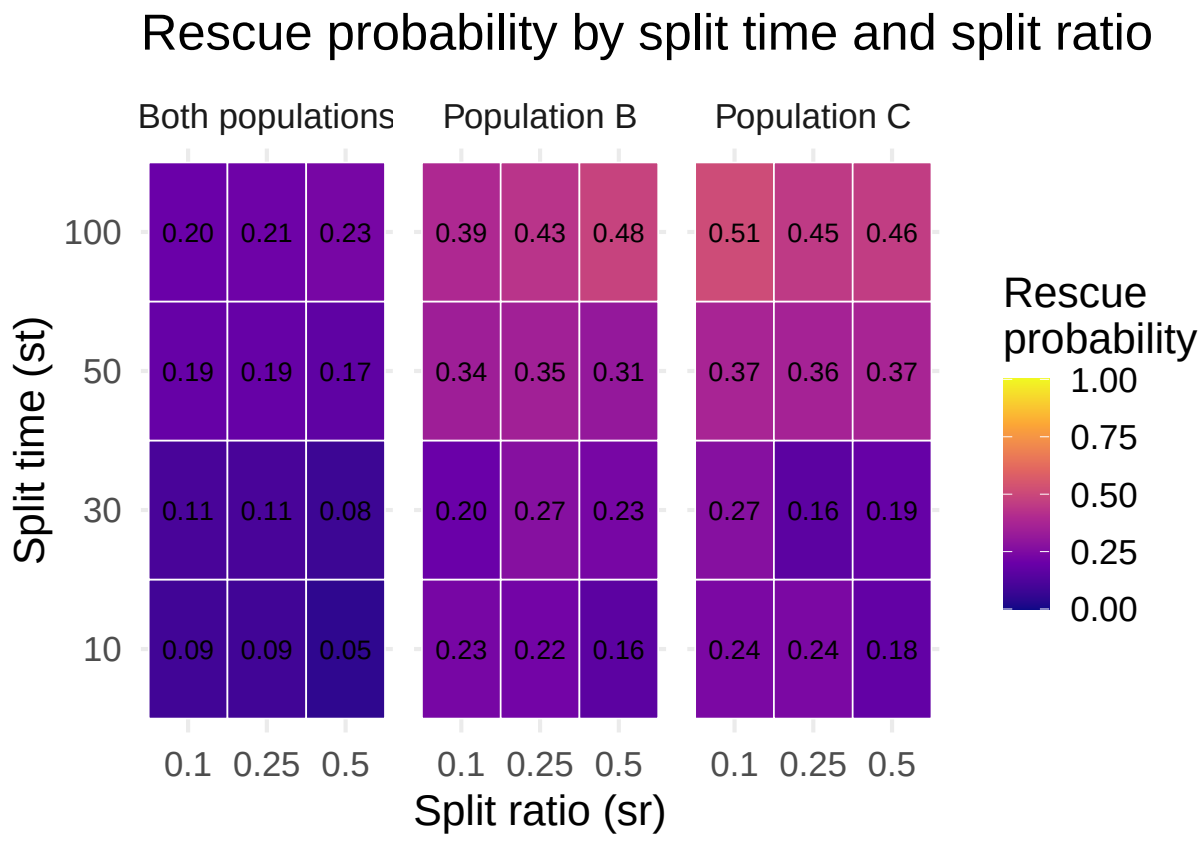
```
# ========================================================================
# ANALYSIS 5: split time and split ratio vary (full grid)
# ========================================================================
```

```r
res5 <- run_scenarios(
  split_times   = c(10, 30, 50, 100),
  habitat_props = 0.5,
  split_ratios  = c(0.1, 0.25, 0.5)
)

long5 <- to_long(res5$prob)

print(res5$prob)

# Heatmap: one panel per population
ggplot(long5, aes(x = factor(sr), y = factor(st),
                  fill = rescue_probability)) +
  geom_tile(color = "white") +
  geom_text(aes(label = sprintf("%.2f", rescue_probability)),
            size = 3.5) +
  facet_wrap(~ population) +
  scale_fill_viridis_c(limits = c(0, 1), option = "C") +
  labs(
    title = "Rescue probability by split time and split ratio",
    x = "Split ratio (sr)",
    y = "Split time (st)",
    fill = "Rescue\nprobability"
  ) +
  theme_minimal(base_size = 16)
```



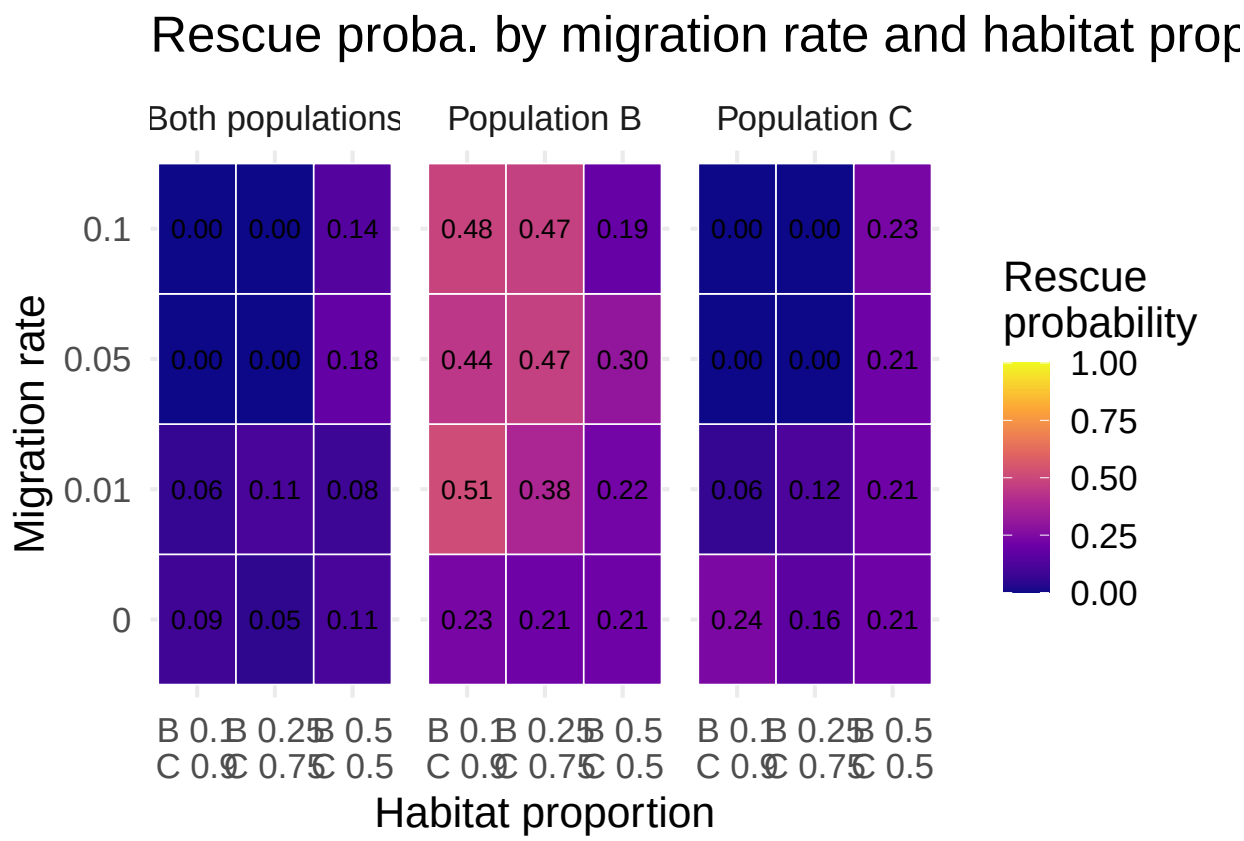
### Split time x split ratio

This Plot combines split time with split ratios of 0.1, 0.25 and 0.5. Rescue probability increased with split time in both habitats. For habitat B it rose from about 0.2 at generation 10 to about 0.4 to 0.5 at generation 100. Habitat C showed the same pattern. Rescue of both habitats also increased, from 0.05 to 0.09 at generation 10 to about 0.2 at generation 100. The split ratio had little effect. Differences between ratios were small and showed no clear direction.

```
# ======================================================================
# ANALYSIS 6: migration rate and habitat proportion vary (full grid)
# ======================================================================
res7 <- run_scenarios(
  split_times   = 30,
  habitat_props = c(0.1, 0.25, 0.5),
  split_ratios  = 0.5,
  migrates      = c(0, 0.01, 0.05, 0.1)
)
long7 <- to_long(res7$prob)
print(res7$prob)

# label showing the proportion of BOTH habitats on two lines
hp_values7 <- sort(unique(long7$hp))
long7 <- long7 %>%
  mutate(hp_label = factor(hp, levels = hp_values7,
                           labels = paste0("B ", hp_values7, "\nC ", 1 - hp_values7)))

ggplot(long7, aes(x = hp_label, y = factor(mig), fill = rescue_probability)) +
  geom_tile(color = "white") +
  geom_text(aes(label = sprintf("%.2f", rescue_probability)), size = 3.5) +
  facet_wrap(~ population) +
  scale_fill_viridis_c(limits = c(0, 1), option = "C") +
  labs(title = "Rescue proba. by migration rate and habitat propo.",
       x = "Habitat proportion", y = "Migration rate",
       fill = "Rescue\nprobability") +
  theme_minimal(base_size = 16)
```




**Migration x habitat proportion**

The last analyses combines the migration rate with habitat proportions of 0.1, 0.25 and 0.5. Without migration, the habitats were rescued about equally often, at every proportion. Even the smallest habitat had no advantage. With migration, unequal habitats behaved very differently. The smaller habitat B was rescued more often. The larger habitat C was rarely rescued at 0.01, at most 0.11 and never at 0.05 or higher. Rescue of both habitats dropped to zero accordingly.

## Discussion

Our simulations suggest that evolutionary rescue remains possible when a habitat is fragmented after the rescue process has begun, but the outcome depends on the timing of the split, the habitat sizes and migration.

**Time point of fragmentation**

The rescue probability tended to increase with later splits. In equally sized habitats, it rose from a probability of 0.24 (B) and 0.23 (C) at generation 10 to about 0.43 (B) and 0.41 (C) at generation 100. This shows that later splits may give mutants more time to arise and establish before the population is divided. However we need to be cautious about this result, because only the populations that survived until the split time were included, which may inflate rescue probabilities for later splits.

### Habitat size

When habitat sizes were unequal, the smaller habitat tended to be rescued more often than the larger one, and this difference increased with later splitting time. Similar results were found in (Bell & Gonzalez, 2011), where they looked at yeast and surprisingly found that adaptation is faster in small populations. At a habitat proportion of 0.1, the rescue probability of habitat B rose from 0.44 to 1.00 with later splits, while habitat C reached at most 0.17. Because the rescue threshold depends on carrying capacity, a smaller habitat needs fewer mutants to count as rescued. Furthermore, when considering the rescue probability based on habitat size, it is important to note that our settings for migration can also influence ER here. Since a fixed percentage of individuals always migrate from one habitat to another, a very large number of individuals will always move from the larger habitat to the smaller one. This is why the carrying capacity of the smaller habitat may be reached more quickly solely due to migration. Its higher rescue probability therefore does not necessarily mean that small habitats are better for long-term population persistence.

### Migration

Migration has a larger effect when habitats are more uneven in size. Because the number of migrants is proportional to the population of the source habitat, the larger habitat sends many more individuals to the smaller one than it receives, so even low migration rates create a noticeable flow. In human-altered landscapes, migration corridors should therefore be recommended. This recommendation should be treated with caution because our results do not show that migration improves rescue overall. Rescue of both habitats dropped to zero at higher migration rates in unequal habitats, so the effects of corridors need further testing.

### Individual distribution

The split ratio had little effect on rescue of either habitat, and differences between ratios showed no clear direction. The initial distribution of individuals therefore appears less important than the timing of the split or the habitat sizes under the conditions tested.

### Biological relevance

Even though our model is based on a theoretical simulation, our project can be applied and used primarily in the field of conservation biology, as it highlights which factors have the most significant impact on a population experiencing habitat fragmentation. At a time when nature is increasingly dominated by humans and habitats are being fragmented as a result, this kind of projects and research form the basic knowledge to understand which factors have to be most considered, whether in the construction of a highway, in supporting measures after a fire, or in the planning of migration corridors in already fragmented habitats, to ensure the survival or growth of the populations.

### Limitations & Improvements

Our rescue measure has an important limitation due to the fact that a habitat only counted as rescued if the number of mutants was at least equal to K in the final generation. Because populations fluctuate around K, some established populations may have been classified as not rescued. The model is also simplified, meaning, that it assumes a haploid population, a single beneficial mutation and only two habitats, and `decay_rate`, `sel_coeff` and `mut_rate` were not varied. In addition, 100 replicates per scenario are too few to interpret small differences as real effects, why a higher number of replicates is recommended.

Future work should test therefore alternative rescue definitions, more replicates, different selection and mutation parameters and more than two habitats. This would show whether the patterns found here are robust and help to assess their relevance for conservation. In addition, when applying this model in other contexts, a real-world connection to an existing species or population should be established. Although our

model simulates the theoretical growth of a population, many confounding factors can still mask the actual effect of habitat fragmentation, as has been demonstrated in (Ewers & Didham, 2006). This is because habitat fragmentation can have different effects depending on various life history traits, such as trophic level, migration ability, or habitat specialization.

### Conclusion

Evolutionary rescue can remain possible when a habitat is fragmented after the rescue process has begun, but the outcome depends on the timing of fragmentation, habitat sizes and migration. Later splits were associated with higher rescue probabilities, and smaller habitats were rescued more often when migration was possible, whereas the distribution of individuals had little effect. We found no evidence that migration improved rescue overall. Because these patterns may partly reflect survivor bias, asymmetric migration and our rescue definition, further simulations are needed before they can inform conservation measures.

**Authorship Contributions** Jan Leuenberger Code: carrying capacity, migration , plotting model, plotting analyses Report: Results, Discussion, Abstract

Safeeya Ramis Code: split time, habitat proportion, split ratio, Analyses Report: Introduction, Methods , Discussion AI promts and pdf knitting

# Appendix

## Additional plots

# References


Baiotto, T., & Guzman, L. M. (2025). Maintaining Local Adaptation Is Key for Evolutionary Rescue and Long-Term Persistence of Populations Experiencing Habitat Loss and a Changing Environment. Evolutionary Applications , 18(3). https://doi.org/10.1111/eva.70081

Bell, G., & Gonzalez, A. (2011). Adaptation and Evolutionary Rescue in Metapopulations Experiencing Environmental Deterioration. Science, 332(6035), 1327–1330. https://doi.org/10.1126/science.1203105

Cheptou, P. O., Hargreaves, A. L., Bonte, D., & Jacquemyn, H. (2017). Adaptation to fragmentation: Evolutionarydynamics driven by human influences. In Philosophical Transactions of the Royal Society B: Biological Sciences (Vol. 372, Number 1712). Royal Society of London. https://doi.org/10.1098/rstb.2016.0037

Ewers, R. M., & Didham, R. K. (2006). Confounding factors in the detection of species responses to habitat fragmentation. In Biological Reviews of the Cambridge Philosophical Society (Vol. 81, Number 1, pp. 117–142). https://doi.org/10.1017/S1464793105006949

Gonzalez, A., Ronce, O., Ferriere, R., & Hochberg, M. E. (2013). Evolutionary rescue: An emerging focus at the intersection between ecology and evolution. Philosophical Transactions of the Royal Society B: Biological Sciences, 368(1610). https://doi.org/10.1098/rstb.2012.0404

Home, R., Angelone, S., Hunziker, M., & Bolliger, J. (2014). Public preferences for ecosystem-enhancing elements in agricultural landscapes in the Swiss lowlands. Journal of Integrative Environmental Sciences, 11(2), 93–108. https://doi.org/10.1080/1943815X.2014.923913